Step 4 — A Reasoner Agent That Only Answers From What the Retriever Found — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 4: Step 3's `reason_node` quotes one excerpt word for word. A real answer often needs several excerpts, rephrased — so a model has to write it. The risk is that the model quietly fills gaps from memory. So the upgraded Reasoner may use **only** the excerpts, must cite each fact's file, and has an exact sentence for "not covered". It stays a separate node, so search quality and answer quality can be checked apart. **If not:** when an answer is wrong, you couldn't tell whether the search found the wrong thing or the writer used the right thing badly.

After this step document questions get real, cited answers. Only `agents/reasoner_agent.py` and a new prompt file change — the graph, the state and every other node stay the same.

Solution — a strict, citing Reasoner with its own prompt

Story — `prompts/reasoner.txt`: three rules — only the excerpts; name each fact's file in brackets; and an **exact** sentence for "not covered". **If not:** an unprompted model tends to fill the gap from memory instead of admitting the excerpts fall short.

`src/documind/prompts/reasoner.txt`

```
You answer questions using ONLY the document excerpts below. After each
fact, name its source file in square brackets, like [refund_policy.md].
If the excerpts do not answer the question, reply exactly:
The documents don't cover this.

Excerpts:
{sources}

Question: {task}
```

Changed from Step 3: `reason_node` keeps its name, its input (the state) and its output (`answer` + its path entry); inside, a Doc05 chain now writes the answer. `format_sources()` is new.

Story — `agents/reasoner_agent.py`: `format_sources()` puts each excerpt's file name in front of it, so the model *can* cite it. The node only writes — it doesn't search and doesn't approve. **If not:** the model would have text but no file names to cite.

```
# src/documind/agents/reasoner_agent.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from documind.models.llm import get_model
from documind.state import DocState
from documind.utils.prompts import load_prompt

# new in Step 4
def format_sources(sources: list[dict]) -> str:
    # why: each excerpt carries its file name, so the model can cite it
    text = ""
    for source in sources:
        text = text + f"[{source['source_file']}] {source['text']}\n\n"
    return text

# changed in Step 4: same name and output, now written by the model
def reason_node(state: DocState) -> dict:
    # why: its own prompt, and it only ever sees the found excerpts — it
    # writes the answer, it doesn't search or approve
    prompt = ChatPromptTemplate.from_template(load_prompt("reasoner"))
    chain = prompt | get_model() | StrOutputParser()
    answer = chain.invoke({
        "sources": format_sources(state["sources"]),
        "task": state["task"],
    })
    return {"answer": answer, "path": ["reason_node"]}
```

Run it: the tests from Step 3 still pass unchanged — `case 1` now shows a written answer. Then check a few answers **by hand** against the documents; "it ran" doesn't prove "it's grounded".

```
You: How long do refunds take to arrive?
  -- needs your approval --
  answer: Refunds are paid back to the original payment method within 5
working days after the returned product arrives [refund_policy.md].
  why:   every answer is checked by a person for now
  Approve? (yes/no): yes
  path: clean_task -> retrieve_node -> reason_node -> human_approved
```

(The answer is wrapped here to fit the page.)